

The redo statement: The *redo* statement in Ruby helps in restarting the iteration of the most internal loop. It helps to restart *yield* or *call* if called within a block, without checking the loop condition. For example:

```
for a in 0..5
  if a < 2 then
    puts "Value is #{a}"
    redo
  end
end
Output
Value is 0
Value is 0
.....
```

This will be an infinite loop.

Arrays

Arrays are like bags that contain things. They comprise groups of objects, of any data type, arranged in a sequence. Arrays are zero based since the index of the first item is always zero.

.each do is a an iterator method which returns all the elements of an array.

Ways of creating arrays: There are several ways to create arrays in Ruby.

```
numerals = [14,15, 16, 17, 18]
fruits = %w( mango grape melon )
bills = Array.new
bills << 4.55 << 3.22 << 3.55 << 8.55 << 3.23
all_datatype = [1, -5,"text", 4.3]
puts numerals.inspect
puts fruits.inspect
puts bills.inspect
puts all_datatype.inspect
```

In the above example ‘%w’ will allow you to add elements in an array without a comma, while ‘inspect’ allows you to view the array object.

Hash

A hash comprises pairs of information, where each pair contains a name or a unique key and a value. Hashes are sometimes called associated arrays. In contrast to arrays, hashes may have arbitrary objects like indexes. There are two ways to create hashes — new keyword and hash literal.

Here’s an example of creating a hash using a new keyword:

```
animals = Hash.new
#adding two pairs of values to the hash
```

```
animals[1] = "Dog"
animals[2] = "Cat"
animals[3]= "Rabbit"
puts animals
#Output : {1=>"Dog", 2=>"Cat", 3=>"Rabbit"}
```

 **Note:** Instead of the square brackets [] you can also use the *store* method for initialising the hash with certain values.

Blocks

Blocks are unidentified pieces of code that accept arguments and return a value. They are useful in passing a piece of code as an argument to a method using *proc*, which will be discussed later.

Also, they can be associated with an array to perform various operations.

For syntax in the blocks, use the following commands:

Single statement: ‘curly braces’
Multiple statements: ‘do and end’

Example: The following code gives you an idea of what a block looks like. Blocks should be associated with an array or method to actually work.

```
#multiple statements
do
  puts "I am Ruby"
  puts "greeting you from inside a block!"
end
#Single statement
{ puts "I am Ruby, greeting you from inside a block!" }
```

Exception handling

- In Ruby, exceptional conditions represent an object, which can either be a descendant or an instance of the *exception* class.
- An exception can be handled by providing a rescue clause, whereby the program control will flow to this clause.
- The predefined classes in Ruby, like *exception* and its children, are used to handle program errors.
- Some of the built-in Ruby sub-classes of exceptions are: NoMemoryError, ScriptError, SecurityError, SignalException, StandardError, SystemExit, SystemStackError and fatal error. **END** 

By: Neetesh Mehrotra

The author works at NIIT Technologies as a senior test engineer and his areas of interest are Java development and automation testing. He can be contacted at mehrotra.neetesh@gmail.com.